

```
In [3]: # install github clone - allows for easy cloning of subdirectories
!pip install github-clone
from pathlib import Path

# clone datasets
if not Path('chapter_8_datasets').is_dir():
    !ghclone https://github.com/neonwatty/machine-learning-refined-notes-assets/tree
else:
    print('chapter_8_datasets already cloned!')

# clone library subdirectory
if not Path('chapter_8_library').is_dir():
    !ghclone https://github.com/neonwatty/machine-learning-refined-notes-assets/tree
else:
    print('chapter_8_library already cloned!')

# clone images
if not Path('chapter_8_images').is_dir():
    !ghclone https://github.com/neonwatty/machine-learning-refined-notes-assets/tree
else:
    print('chapter_8_images already cloned!')

# clone videos
if not Path('chapter_8_videos').is_dir():
    !ghclone https://github.com/neonwatty/machine-learning-refined-notes-assets/tree
else:
    print('chapter_8_videos already cloned!')

## run universally whether in collab or local ##
# append path for local library, data, and image import
import sys
sys.path.append('./chapter_8_library')
sys.path.append('./chapter_8_images')
sys.path.append('./chapter_8_datasets')

# import section helper
import section_8_3_helpers

# dataset paths
dataset_path_1 = 'chapter_8_datasets/2d_span_data.csv'
dataset_path_2 = 'chapter_8_datasets/3d_span_data.csv'

# image paths
image_path_1 = "chapter_8_images/pca_classic.png"
image_path_2 = "chapter_8_images/Fig_9_7.png"

# standard imports
import matplotlib.pyplot as plt
from IPython.display import Image
import autograd.numpy as np
from matplotlib import gridspec

# this is needed to compensate for matplotlib notebook's tendency to blow up images
%matplotlib inline
```

```
from matplotlib import rcParams
rcParams['figure.autolayout'] = True

%load_ext autoreload
%autoreload 2
```

Requirement already satisfied: github-clone in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (1.2.0)
 Requirement already satisfied: requests>=2.20.0 in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (from github-clone) (2.32.4)
 Requirement already satisfied: docopt>=0.6.2 in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (from github-clone) (0.6.2)
 Requirement already satisfied: charset_normalizer<4,>=2 in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (from requests>=2.20.0->github-clone) (3.4.4)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (from requests>=2.20.0->github-clone) (2.2.3)
 Requirement already satisfied: certifi>=2017.4.17 in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (from requests>=2.20.0->github-clone) (2026.1.4)
 Requirement already satisfied: idna<4,>=2.5 in /mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/lib/python3.8/site-packages (from requests>=2.20.0->github-clone) (3.11)
 WARNING: You are using pip version 21.1.1; however, version 25.0.1 is available.
 You should consider upgrading via the '/mnt/c/Users/hugoa/Programming/ML-DL/ML-optimization/.venv/bin/python3.8 -m pip install --upgrade pip' command.
 chapter_8_datasets already cloned!
 chapter_8_library already cloned!
 chapter_8_images already cloned!
 chapter_8_videos already cloned!

```
In [7]: def autoencoder_cost(w):
        C = w[0]
        W = np.dot(C.T, X)

        cost = np.sum((np.dot(C, W) - X)**2)
        return cost / float(X.shape[1])
```

```
In [8]: # Load in dataset
X_original = np.loadtxt(dataset_path_2, delimiter=',')

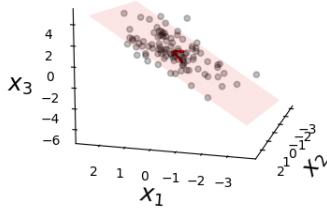
# center data
X_means = np.mean(X_original, axis=1)[:, np.newaxis]
X = X_original - X_means

# run gradient descent to minimize the Least Squares cost for linear regression
g = autoencoder_cost; max_its = 100; alpha_choice = 0.01;

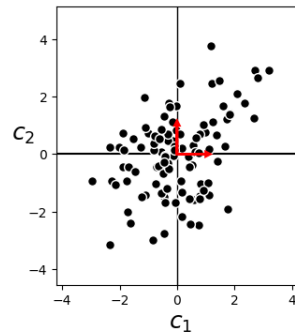
# set up initialization
scale = 0.1;
C = scale*np.random.randn(3,2)
w = [C]
# run gradient descent
weight_history, cost_history = section_8_3_helpers.gradient_descent(g, alpha_choice, m
```

```
In [10]: # produce visuals
C = weight_history[-1][0]
view = [15, 105]
section_8_3_helpers.project_data_from_3d_to_2d(X, C, view)
```

Original data



Encoded data



Decoded data

